



Tutorial 7 | OS Monsoon'25

Section A

| Author → [Rachit Bhandari](#)



EGOS-2000 Operating System



GitHub Repo → <https://github.com/yhzhang0128/egos-2000/>

Introduction to EGOS-2000

EGOS-2000 is a minimalist, open-source operating system built in roughly **2000 lines of code**.

- Target architecture → **RISC-V**
- Design goal → Serve as an **educational tool** for learning fundamental OS concepts in a simplified yet realistic way.

Why EGOS-2000?

- **Simplicity** → only three layers, easy to grasp.
- **Educational clarity** → demonstrates concepts like memory management, scheduling, and system calls.
- **Modular design** → its layered structure makes experimentation and modification straightforward.
- **Core OS concepts** → covers the essentials, i.e. interrupts, paging, process scheduling, IPC, file systems.

Layered Architecture of EGOS-2000

EGOS-2000 divides its functionality into **three layers**, each with a distinct role

Earth Layer (Hardware-Specific)

- Handles **direct hardware interactions**.
- Implements device drivers, memory paging, and interrupt handling.
- Provides the **foundation** of the OS.

Grass Layer (Hardware-Independent Services)

- The **operating system services** layer.
- Manages **system calls**, process scheduling, and inter-process communication (IPC).
- Acts as a **bridge** between Earth (hardware) and Application (user-facing).

Example →

A user application requests a file read.

- The **Application Layer** issues a system call.
- The **Grass Layer** interprets this call, then invokes the **Earth Layer's disk driver** to retrieve the data.
- The Grass Layer then returns the data back to the application.

Application Layer (User-Facing)

- Contains **user processes, file systems, and the shell**.
- Offers a command-line interface (CLI) with basic commands →

- `pwd`, `cd`, `ls`, `cat`, `echo`
- Relies on Grass for system services, which in turn may call Earth.

Earth Layer

Earth Layer

Hardware Specific

- **earth/dev_disk**: ROM and SD card
- **earth/dev_page**: memory paging
- **earth/dev_tty**: keyboard input and tty output
- **earth/cpu_intr**: interrupt and exception handling
- **earth/cpu_mmu**: memory paging and address translation
- **earth/cpu_timer**: control registers for timer
- **earth/bus_gpio.c** and **earth/bus_uart.c**: buses
- **earth/earth.c** and **earth/earth.s**: used for initialization
- **earth/earth.ld**: specifies memory layout
- **earth/sd/sd.h**: provides basic definitions
- **earth/sd/sd_init.c**: initializes the SD card
- **earth/sd/sd_rw.c**: provides SD card read and write
- **earth/sd/sd_utils.c**: provides helper functions

The **Earth Layer** is closest to hardware. Its responsibilities include →

Device Drivers

- `dev_disk` → Handles disk/SD card read & write.
- `dev_tty` → Keyboard input, terminal (TTY) output.

TTY (Teletype) Output refers to the way an operating system or program interacts with the terminal or command-line interface, where it displays text based output to the user.

Memory Management

- `dev_page` → Implements paging.
- `cpu_mmu` → Manages virtual-to-physical address translation.

Interrupts and Timers

- `cpu_intr` → Handles hardware interrupts.
- `cpu_timer` → Configures timers for scheduling.

Bus Communication

- `bus_gpio` , `bus_uart` → Communication with peripherals.

This layer is initialized first during boot, preparing the OS for higher-level functions.

Grass Layer

Grass Layer

Hardware Independent

- **grass/syscall**: system call interfaces to user applications
- **grass/process**: data structures for managing processes
- **grass/kernel**: preemptive scheduling and inter-process communication
- **grass/grass.c** and **grass/grass.s**: used for initialization
- **grass/grass.lds**: specifies memory layout

The **Grass Layer** abstracts away hardware and provides the **core OS functionalities** →

- **System Calls** → interface for file I/O, process creation, etc.
- **Process Management** → process control blocks (PCBs), preemptive scheduling.
- **Inter Process Communication (IPC)** → shared memory, message passing, and signals.

It ensures user applications can access services **without directly touching the hardware**.

Application Layer

Application Layer

- **app/system/sys_proc**: allocation and free of user processes
- **app/system/sys_file**: the inode layer over the SD card
- **app/system/sys_dir**: the directory layer over the inode layer
- **app/system/sys_shell**: interactive user interface
- **shell commands**: `pwd`, `cd`, `ls`, `cat`, `echo`, `clock`, `killall`

The **Application Layer** is what users interact with.

Components

- **Process Management (`sys_proc`)** → Handles process creation and termination.
- **File System:**
 - `sys_file` → Implements the **inode layer** (metadata for files).
 - `sys_dir` → Directory management.
- **Shell (`sys_shell`)** → The CLI environment where commands are executed.

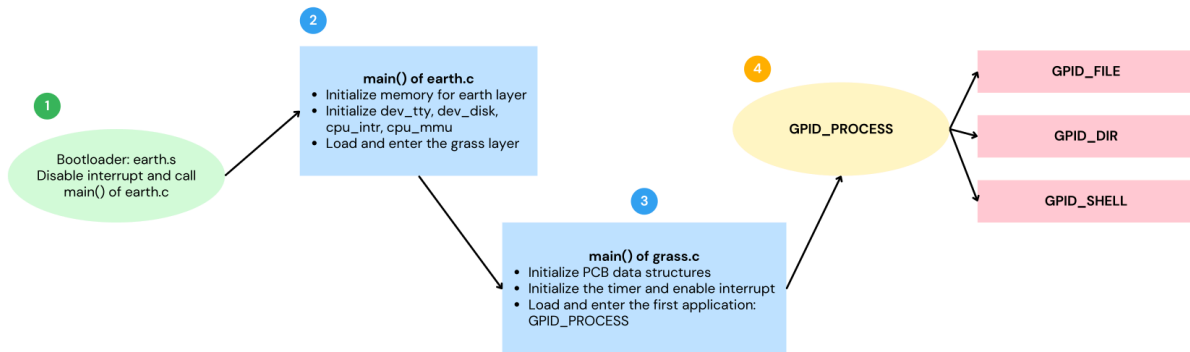
Shell Commands

- `pwd` → Print working directory.
- `cd` → Change directory.
- `ls` → List files.
- `cat` → Display file contents.
- `echo` → Print messages.
- `clock` → Show system clock.
- `killall` → Terminate all processes.

Booting and Initialization Process

Booting and Initialization

Key Files: earth.S, earth.c and grass.S and grass.c



Booting EGOS-2000 involves **layer-by-layer initialization**

Step 1 → Bootloader (Earth Layer)

- File: `earth.s`
- Actions →
 - **Disables interrupts** during early setup. (to prevent any premature or unexpected interruptions during the critical initialization phase)
 - Calls `main()` in `earth.c`. (transfer of control)

Step 2 → Earth Layer Initialization

- File: `earth.c`
- Initializes →
 - **TTY** (keyboard/console)
 - **Disk** (storage I/O)
 - **Interrupt controller** (`cpu_intr`)
 - **MMU** (memory translation)
 - **Timer** (for scheduling)
- Then **loads the Grass Layer**.

Step 3 → Grass Layer Initialization

- File: `grass.c`
- Initializes →
 - **PCBs** (process structures)
 - **Timer** and re-enables interrupts
 - **Scheduler** for process management
- Launches the **first application process**: `GPID_PROCESS`

Step 4: First User Applications

From here, essential system processes are started

- **GPID_PROCESS** → First user-level process. (entry point for launching other essential processes)
- **GPID_FILE** – File operations. (handling interactions with the file system and providing file services to other applications)
- **GPID_DIR** – Directory management. (allowing the system to navigate, read, and modify directory structures)
- **GPID_SHELL** – Launches the shell interface. (primary user interaction point)

Compilation and Setup



Follow → <https://github.com/yhzhang0128/egos-2000/blob/main/USAGES.md>

Step 1 → Setup the compiler and compile egos-2000

Setup your working directory and name it as `$EGOS`.

```
export EGOS=/home/yunhao/egos
cd $EGOS
git clone <https://github.com/yhzhang0128/egos-2000.git>
```

Download the pre-built binaries of the RISC-V GNU compiler toolchain from xPack to `$EGOS` and compile egos-2000.

```
cd $EGOS
tar -zxvf xpack-riscv-none-elf-gcc-14.2.0-3-{linux,darwin}-{x64,arm64}.tar.gz
export PATH=$PATH:$EGOS/xpack-riscv-none-elf-gcc-14.2.0-3/bin
cd $EGOS/egos-2000
make
....
```

Step 2 → Run egos-2000 on the QEMU emulator

Download the [pre-built binaries of QEMU](#) from xPack.

```
cd $EGOS
tar -zxvf xpack-qemu-riscv-8.2.2-1-xxxxxx.tar.gz
export PATH=$PATH:$EGOS/xpack-qemu-riscv-8.2.2-1-xxxxxx/bin
cd $EGOS/egos-2000
make qemu
----- Simulate on QEMU-RISCV -----
qemu-system-riscv32 -nographic -readconfig tools/qemu/config.toml
[CRITICAL] --- Booting on QEMU with core #4 ---
[INFO] Set the CS pin to HIGH and toggle clock
[INFO] Set the CS pin to LOW and send cmd0 to SD card
[INFO] Check SD card type and voltage with cmd8
[INFO] SD card replies cmd8 with status 1
[SUCCESS] Finished initializing the tty and disk devices
[CRITICAL] Choose a memory translation mechanism:
Enter 0: page tables
Enter 1: software TLB
```



Message Passing Interface (MPI)

What is MPI?

MPI (Message Passing Interface) is a standardized and portable message-passing system built for **parallel computing**.

Think of MPI as the “postal service” for processes → it delivers messages between different processes running on multiple cores or even across different machines in a cluster.

With MPI, separate processes can →

- **Send** messages
- **Receive** messages
- **Work together** like a single powerful virtual computer

Why Do We Need MPI?

1. Parallelism for Performance

Big tasks like simulations, scientific computations, or training large machine learning models often overwhelm a single computer. MPI allows many processors to collaborate, splitting the workload.

2. Distributed Systems

In real distributed setups, each processor has its own memory. MPI helps them talk to each other, share results, and coordinate, thus acting like one big system.

3. Scalability

Whether you're using a tiny cluster of 4 computers or a supercomputer with thousands of nodes, MPI scales up (and down) efficiently.

Core MPI Functions

1. `MPI_Init()`

- Initializes the MPI environment.
- Must be called **before any other MPI function**.
- Sets up data structures and prepares processes for communication.

2. `MPI_Comm_rank(MPI_COMM_WORLD, &rank)`

- Returns the **rank (ID)** of the calling process in a communicator.
- Think of the rank as a process's **seat number** on the MPI bus.

3. `MPI_Comm_size(MPI_COMM_WORLD, &size)`

- Returns the **total number of processes** in the communicator.
- Together with rank, it helps each process know **its role** in the group.

4. `MPI_Send(buf, count, datatype, dest, tag, comm)`

- Sends a message to another process.
- Key parameters:
 - `buf` : Data to send
 - `count` : Number of elements
 - `datatype` : Data type (e.g., `MPI_INT`)
 - `dest` : Destination process rank
 - `tag` : Label to identify the message
 - `comm` : Communicator (often `MPI_COMM_WORLD`)

5. `MPI_Recv(buf, count, datatype, source, tag, comm, status)`

- Receives a message from another process.
- Key parameters:
 - `buf`: Buffer to store incoming data
 - `count`: Number of elements expected
 - `datatype`: Type of data expected
 - `source`: Rank of sender process
 - `tag`: Label matching the message
 - `comm`: Communicator
 - `status`: Info about the received message

6. `MPI_Finalize()`

- Cleans up the MPI environment.
 - Called at the **end of the program** to shut everything down gracefully.
-

**When you're done with your
presentation but nobody starts
clapping**

